Grounded Copilot: How Programmers Interact with Code-Generating Models

Shraddha Barke, Michael B. James, Nadia Polikarpova

Proc. ACM Program. Lang., Vol. 7, No. OOPSLA1, Article 78. April 2023.

Paper Review by Taha Cheema - 21204729

The authors present a qualitative study of how programmers interact with GitHub Copilot. To this end, they apply grounded theory, a qualitative research method in which the researchers begin with raw observations and iteratively tag them with categories that emerge from the data itself, rather than starting from a predefined hypothesis. They observed 20 participants (15 from academia, 5 from industry) with a mix of prior Copilot experience as they solved programming tasks across four languages (Python, Rust, Haskell, and Java). They iterated this until a coherent theory was obtained. They then cross-checked the theory with a quantitative analysis of the in-study data and with five livestream videos of programmers using Copilot on YouTube and Twitch. The authors' concluding remarks are that programmer interactions with Copilot fall into two distinct modes. The first is acceleration: the programmer knows what to do and uses Copilot to get there faster using auto-complete. The second is exploration: the programmer is unsure and uses Copilot to scope out their options by circling between different long form code completions (a user can view up to 10 suggestions for the code block). The paper concludes that future programming assistants should be designed with this bimodality in mind.

Since this paper was published in April 2023, coding assistants have evolved drastically. The authors also pointed out that future changes may make their findings obsolete. However, I think there are still important takeaways that apply even now. Firstly, over-reliance on AI suggestions can lead to debugging time sinks later. One of the study's participants said, "if there's any chance that Copilot is going to get it wrong, I'd rather just get it wrong myself, at least that way I understand what's going on". Today, the 'acceleration' of auto-complete has evolved into 'orchestration' of agentic coding assistants to *accelerate* major code changes or even bootstrap apps in hours, sometimes only interacting with these agents using natural language prompts. However, the underlying problem remains: when you don't write the code yourself, you often do not know where to look when you see unwanted behaviour. I also think this can lead to exhaustion in programmers which is ironic given how AI is supposed to be a productivity boost. Secondly, the idea of exploration using is even more common today as AI has become better at reasoning with enhanced context windows. Where earlier, exploration was more about seeing different code snippets, it is now sometimes the exploration of entire project decisions like framework choice, database decisions and so on. In summary then, 'acceleration' and 'exploration' have changed since this paper was published but it might be better to think of them as transmutations.

Having said that, I do have some concerns with the authors' methodology. Firstly, 11 of the 20 participants had not used Copilot before the study and only received a 5 minute training task to familiarize themselves. This is a major threat to validity as humans may greatly adapt their usage patterns if they were using it on the daily for their job (as many in the industry do). However, a larger concern I have is about their grounded theory methodology to begin with. The authors wanted to build a theory from the ground up. However, their conclusions seem to coincide neatly with Daniel Kahneman's 'Thinking, Fast and Slow' framework of System 1 and System 2 cognition (fast and slow cognition) which is mentioned in the 'Summary of findings' section of the introduction. They also refer to System 1 and 2 in Section 2 while giving examples of Copilot-assisted coding. We may wonder if their approach was biased by this pre-set idea of cognition. A frame-agnostic researcher might have surfaced more than two modes from the same observations; for example, a distinct debugging mode, which the paper observes but folds into exploration.

References

Kahneman, D. (2013). *Thinking, Fast and Slow.* Farrar, Straus and Giroux.